COLLEGE ENGINEERING PROJECT · PROJECT 22 OF 55

AI-Based Customer Complaint Classification and Priority Prediction Using NLP

Domain: NLP & Customer Experience Automation | **Module:** project_22_complaint_classification

PROJECT ABSTRACT

Customer complaints provide valuable information about problems faced by users, but handling a large number of complaints manually can be time-consuming and inefficient. Complaints received through emails, online forms, chat platforms, and customer-support systems may contain different types of issues, making it difficult for support teams to quickly identify which complaints require immediate attention. The proposed AI-Based Customer Complaint Classification and Priority Prediction System Using Natural Language Processing (NLP) is designed to automatically understand, categorize, and prioritize customer complaints. The proposed system is developed using Python, NLP techniques, and machine learning algorithms to analyze the textual content of complaints. The system processes the complaint, identifies important textual patterns, and classifies it into relevant categories such as payment issues, product complaints, delivery problems, technical issues, account-related problems, or service dissatisfaction. This reduces the need for support staff to manually read and categorize every complaint. A major uniqueness of the proposed system is the combination of complaint classification and priority prediction in a single intelligent workflow. After identifying the complaint category, the system analyzes the seriousness and potential impact of the issue to predict an appropriate priority level such as low, medium, high, or critical. For example, a general product inquiry may receive a lower priority, while a complaint involving failed payment, account access, or a service interruption can be identified as requiring faster attention. The system can also identify important words and contextual patterns within complaints instead of depending only on predefined keywords. This allows similar complaints to be classified even when customers describe the same problem using different words or sentences. The model can be trained using historical complaint data so that its classification and priority prediction capabilities can improve based on previously handled cases. Another useful feature is the generation of a structured complaint summary, which can help support teams quickly understand the issue without reading the complete customer message. The system can display the complaint category, predicted priority, and relevant information in an organized format. With further development, it can be integrated with customer-support platforms to analyze incoming complaints automatically and support real-time complaint routing. The proposed system can be applied in e-commerce, banking, telecommunications, education, healthcare services, and other organizations that manage large volumes of customer feedback. By automatically identifying both what the complaint is about and how urgently it should be handled, the system can help organizations reduce response time and improve customer-service efficiency. Overall, this project demonstrates how NLP and machine learning can convert unstructured customer

> complaints into actionable support information. The combination of automated classification, priority prediction, contextual analysis, and structured complaint handling provides a practical AI-based approach for building faster, smarter, and more efficient customer-support systems.

📌 Project Metadata & Specifications

Project Title	AI-Based Customer Complaint Classification and Priority Prediction Using NLP
Domain / Specialization	NLP & Customer Experience Automation
Source Directory	`project_22_complaint_classification`
Backend REST API	`http://localhost:5022` (Flask Python)
Frontend Interface	Interactive Dashboard (`frontend/index.html`)
Execution Mode	Hybrid Dual-Mode (Online API & Offline Autonomous Simulation)

🏗️ System Methodology & Implementation Phases

PROJECT 22 — PHASE 1

Phase 1 — Requirements, Data and Prototype

The first phase will convert the project concept into a measurable prototype for healthcare/medical AI. The work begins by defining the exact input, output, user flow and evaluation criteria from the project abstract. A small requirements specification will identify the primary prediction/detection task, expected classes or target value, acceptable response time, and the form in which results must be displayed.

1. Problem Definition and Scope

Define the operational scenario, target users, system boundaries, assumptions, success criteria and non-functional requirements. Separate what the model must predict from what the application must display or recommend. Identify cases that should be rejected, flagged for review, or returned with low confidence.

2. Data Preparation

Identify a suitable dataset or approved collection procedure matching the abstract. Build a data dictionary, remove unusable records, handle missing values, normalize or standardize numerical fields where appropriate, and verify labels. For image/video/audio/text projects, define preprocessing, augmentation and quality-control rules. Split data into training, validation and test sets without leakage.

3. Baseline Model and Experiment Setup

Implement a reproducible baseline using the algorithm/model family stated in the abstract when one is specified. Establish a simple reference result before introducing additional complexity. Record dataset version, preprocessing settings, model configuration, random seed, training time and evaluation metrics so that later improvements can be compared objectively.

4. Initial Application Prototype

Build the minimum user-facing workflow needed to demonstrate the core task. The prototype should accept valid input, run preprocessing, invoke the baseline model, and return the predicted result with an

understandable confidence/score where supported. Keep model logic separate from the interface so the model can be replaced without rewriting the application.

Phase 1 Deliverables

Requirements and scope note; prepared dataset or data pipeline; baseline model; experiment log; initial application/prototype; and a short Phase 1 validation report documenting what works, what remains uncertain, and the measurements to be improved in Phase 2.

Project-Specific Phase 1 Work Items

- Define text cleaning, tokenization/embedding and document-length handling rules.
- Create representative examples for each output class or information field.
- Track confidence and ambiguous cases for human review rather than forcing uncertain outputs.

PROJECT 22 — PHASE 2

Phase 2 — Optimization, Integration and Validation

The second phase will turn the Phase 1 prototype into a validated project for healthcare/medical AI. The emphasis is on improving model reliability, connecting the complete workflow, testing realistic edge cases, and producing evidence that the final system meets the defined objectives.

1. Model Improvement

Compare relevant model configurations, tune important hyperparameters, address class imbalance where present, and evaluate whether additional features or augmentation improve generalization. Use validation data for tuning and reserve the test set for final evaluation. Prefer measurable improvements over unnecessary model complexity.

2. End-to-End Integration

Connect the data/input layer, preprocessing, inference engine, result-processing logic, database or file storage where required, and user interface. Add input validation, error handling, logging and clear status messages. If recommendations, alerts, summaries, risk scores or dashboards are part of the abstract, implement them as a separate decision-support layer around the model.

3. Testing and Evaluation

Perform functional, model, integration and usability tests. Evaluate accuracy or regression error as appropriate, along with precision, recall, F1, confusion matrix, ROC-AUC, latency or other task-specific measures where relevant. Test difficult, noisy, incomplete and out-of-distribution inputs and document failure cases instead of hiding them.

4. Demonstration and Final Evidence

Prepare representative test cases, screenshots or result tables, model-comparison evidence and a final demonstration workflow. Document limitations, data constraints, privacy/security considerations where applicable, and future extensions. The final report should distinguish measured results from proposed future work.

Phase 2 Deliverables

Optimized model; integrated application; test and evaluation report; representative evidence; limitations and risk register; deployment/demo instructions; and final project documentation suitable for academic review or demonstration.

Project-Specific Phase 2 Validation

- Treat predictions as decision-support outputs and document that they require appropriate human/clinical review.
- Evaluate class-wise performance so weaker disease or risk categories are visible.
- Test missing/noisy patient inputs and ensure uncertain predictions are clearly identified.

Final Expected Outcome

A reproducible, tested prototype that implements the project concept described in the supplied abstract, together with measurable evaluation evidence and clear documentation of limitations, assumptions and future enhancement opportunities.

🚀 Local Deployment & Execution Guide

Follow the steps below to initialize and demonstrate the system:

1. Backend Server Setup

```
# Navigate to the backend directory cd "/Users/balamurugan/Documents/orbyte globel
/college project/project_22_complaint_classification/backend" # Install project
dependencies pip install -r requirements.txt # Run the Flask API service python3 app.py
```

The backend microservice will listen for requests at `http://localhost:5022`.

2. Frontend User Interface

- Open `/Users/balamurugan/Documents/orbyte globel /college project/project_22_complaint_classification/frontend/index.html` directly in Google Chrome, Firefox, or Safari.
- Alternatively, launch via the central **Master Launchpad Portal (`index.html`)** located at the root directory.
- The user interface is designed with **hybrid dual-mode resilience**: when the backend API is live, real-time inferences are queried; when offline, authentic client-side mathematical models simulate sensor telematics.